

Abstract: In this assignment, three plugins were created in JUCE: a sequential IIR filter EQ using the `juce::IIRFilter` class; an equivalent implementation using the JUCE DSP module `ProcessorChain`; and a parallel band-splitting EQ using lowpass, bandpass, and highpass filters. Implementation errors were then identified during testing and then corrected. Finally, the audio output of white noise from all three plugins was inspected visually using the spectrum function on Sonic Visualiser, and the spectral differences between the implementations were then analysed.

Step 1: As a first step, a simple filter plugin was generated in JUCE, using the `juce::IIRFilter` class, with reference to code written during a lab. This plugin has 2 bands of filters (low shelf and high shelf), with Frequency, Gain, and Quality Factor (Q) parameters for both high and low frequencies. This plugin can also generate white noise or load external audio for testing.

Step 2: Once built, the Graphical User Interface (GUI) for this filter showed low frequency parameters on the right, and high frequency parameters on the left. To swap these, as per convention, the `FiltersLab2AudioProcessorEditor::resized()` function of the `PluginEditor.cpp` was adapted to move the low frequency parameters to the left, and the high frequency parameters to the right.

Step 3: To add parameters to control the middle frequencies of the equaliser, two new filters were created, one for low mids, and one for high mids. To do this, every instance where the current parameters were being executed throughout the code were adapted by adding 'low mid' and 'high mid' parameters, in the same format and in the same sections where there was code for the high and low frequencies. Most crucially, this was in this section:
`std::vector<std::unique_ptr<juce::RangedAudioParameter>> parameters` of the `PluginProcessor.cpp`; Here, four EQ bands were defined with the following approximate frequency ranges, and centre frequencies:

Low Frequency bandwidth: 0-400 Hz, centre frequency: 200 Hz

Mid Low Frequency bandwidth: 400-1 kHz, centre frequency: 700 Hz

Mid High Frequency bandwidth: 1-6 kHz, centre frequency 3.5 kHz

High Frequency bandwidth: 6-20 kHz, centre frequency: 13 kHz

EQs often space bandwidth ranges in a logarithmic scale, to match human perception of frequency (Reiss & McPherson, 2015, p. 96). However, the above ranges were chosen to maximise the perceptual difference between bands on the white noise signal, for this four-band design. The gain range was also adjusted for each parameter to between +/- 10. This enables the ability to cut frequencies as well as boost them. Manipulating the low frequencies can manipulate bass (this could include cutting low hums, for example); mid frequencies can be used to adjust the level of particular instruments (such as the guitar or piano in a mix); and high frequencies can be used to cut or boost sharp sounds (like cymbals or sibilants). Furthermore, the Q range was set to between 0-1 for each parameter to provide broad filters. Q controls the width of the frequency band, whereby higher Q values produce narrower filters, and lower Q values produce broader responses. In many filter designs, $Q = \text{Centre frequency} / \text{bandwidth} = \omega_c/B$.

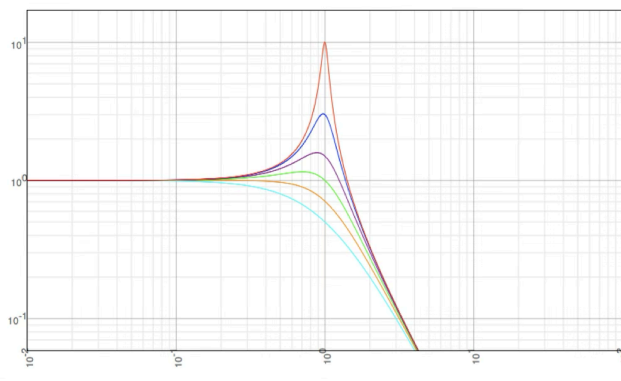


Figure 1: Comparison of Q sizes for a low pass resonator filter¹

¹ Taken from lecture slides: Reiss, J. *Filters and EQ in JUCE* [PowerPoint slides].

https://github.com/joshreiss/Audio_Effects-Theory_Implementation_and_Application/tree/main/04%20-%20Filter%20effects/A%20-%20Filtering (accessed 11/03/26)

During implementation, a glitch was found in the code. Whenever a parameter was set to 0, the plugin would crash unexpectedly. This was due to this assertions like this, which prevents any frequency parameter from reaching 0:

```
jassert (cutOffFrequency > 0.0 && cutOffFrequency <= sampleRate * 0.5);
```

The parameters were therefore adjusted so that they could never reach 0, with 0.01 being the smallest possible value.

Step 4: The GUI was then tidied with a few tweaks. Chat GPT was used to help with the formatting and appearance. The overall panel size was adjusted in this line: `setSize (750, 500);` and the margin between areas was also changed to: `reduced(20,0)` in the PluginEditor.cpp. This gave the following GUI, which looks more neat and professional:

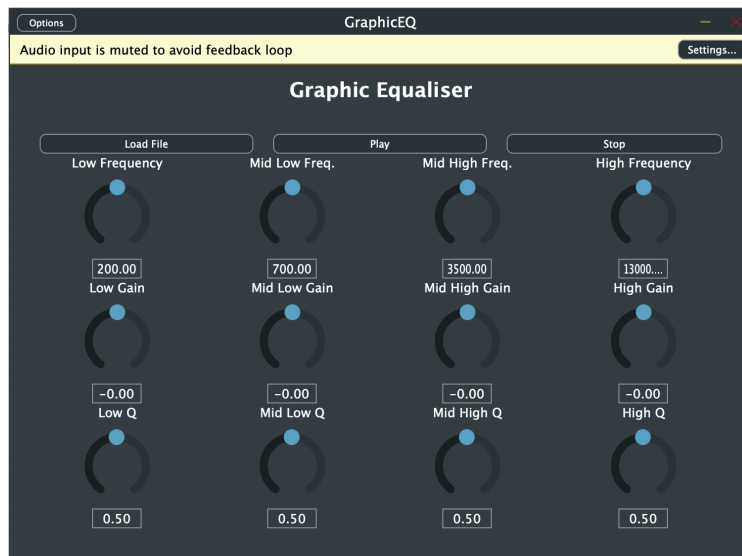


Figure 2: GUI for my Graphic Equaliser

Step 5: Note that since the code connects the filters sequentially, the white noise passes through each band in series, producing a cumulative EQ response. This output was then recorded using QuickTime with gain turned down to -10 (and Q set at 0.5) for each frequency parameter. The result was then inspected through a layered spectrum on Sonic Visualiser. This clearly shows the effect of the different filter parameters:

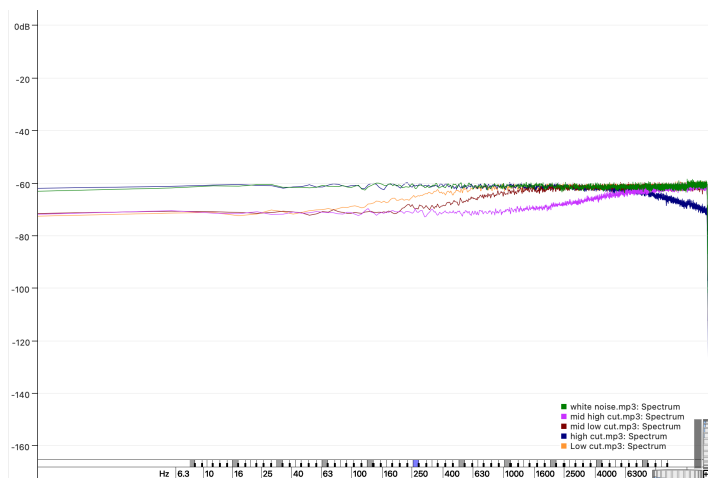


Figure 3: Layered spectra of different filter parameters

However, this visual inspection raised a question about these filters. It appears as if the mid low cut (in red) and the mid high cut (in pink) were acting as a low shelf (albeit with different frequency bandwidths). In other words, they were cutting out low frequencies to the same degree as the low cut frequency bandwidth. By inspecting the code, the filter was found to be set to `makeLowShelf` instead of `makePeakFilter` as a result of copying and pasting the low cut filter. This code was therefore adjusted and the spectra was recreated on Sonic Visualiser, which clearly shows the intended effect of these parameters:

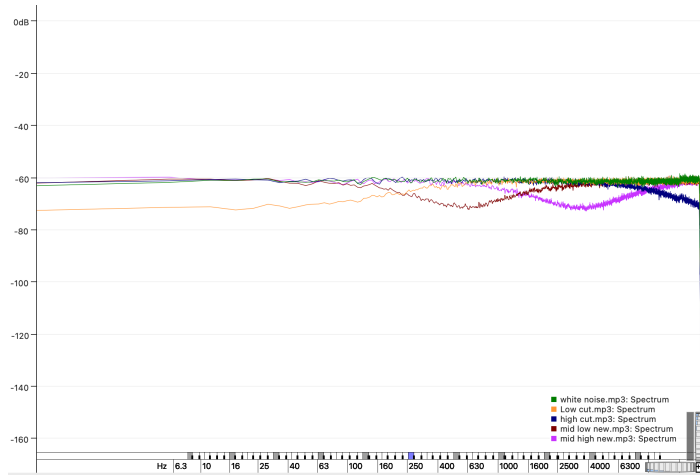


Figure 4: Layered spectra of different (altered) filter parameters

Extra credit 1: A ProcessorChain was then implemented, which keeps the GUI and sound output identical, but makes the code simpler and easier to scale (for example, it enables easier addition of new frequency bands). Previously, each band was computed via a separate ProcessorDuplicator but in this revision, there are now four `ProcessorDuplicator<IIR::Filter>` elements, allowing for the EQ to be processed as a single DSP. The chain is prepared by `juce::dsp::ProcessSpec spec;` and the filter coefficients are updated to `*eqChain.get<index>().state` in the processBlock. Finally, the audio buffer is wrapped in `juce::dsp::AudioBlock` and processed with `eqChain.process(context);`. Using the ProcessorChain simplifies the DSP structure, and therefore makes it easier to maintain and to replicate. These files can be found in the folder under 'Part 2 - ProcessorChain'. As seen from this graph, these filters, despite being built by different code, produce the same results as the original. To illustrate the differences in boost and cut, the gain was also boosted in the mid low and high frequency parameters:

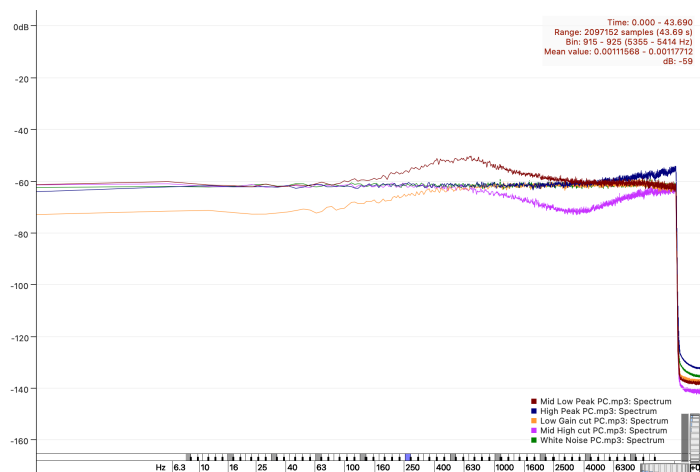


Figure 5: Layered spectra of different filter parameters, using the ProcessorChain implementation

Extra credit 2: A parallel filter structure using lowpass, bandpass, and highpass filters was then implemented. The input audio was first copied into a temporary buffer and then routed to each filter band independently. This differs from the previous implementations, where each band was processed sequentially. Here, each band is processed independently before being recombined for the output. A similar architecture is used for loudspeaker crossover systems, where the woofer receives a lowpass filtered signal, the midrange receives the bandpass filtered signal, and the tweeter receives the highpass filtered signal (Reiss & McPherson, 2015, p. 87). Each frequency band uses `*juce::dsp::IIR::Coefficients<float>` as well as a gain processor to allow amplification, converted to linear gain, which is given by: `Band.gain.setGainLinear(std::pow(10.0f, BandGainDb / 20.0f))`. The processed bands are then summed together to produce the output signal, using `buffer.addFrom` which reconstructs the signal by summing the band-limited components. These files can be found in the folder under 'Part 3 - LPF_BP_HPF'.

White noise was then recorded from this filter, with the gain for each parameter set to -10, and the Q set to 0.5. Perceptually, the white noise generated by this version (without any cut or boost) sounds as though some of the higher frequencies are slightly attenuated, making the signal more muffled. You can see from the plots below in Figure 6 why there is a perceptual difference: there is a dip in frequency at around 11,000 Hz. This is because the summed parallel band-splitting does not perfectly reconstruct the original spectrum, as the filters do not crossover exactly (i.e they do not form a 'true' allpass reconstruction when summed, as they are not ideal filters). Specifically, the mid high bandpass (centred at around 3.5 kHz) had declined by around 10-12 kHz, and the highpass filter (centred at 13 kHz) had not yet fully risen at the 10-12 kHz range. Phase differences between the bandpass and highpass filters may also be causing some destructive interference when the signals are summed. These properties of the parallel filter therefore produce some small dips in the spectrum. This shows the differences between parallel band-splitting and graphical EQ.

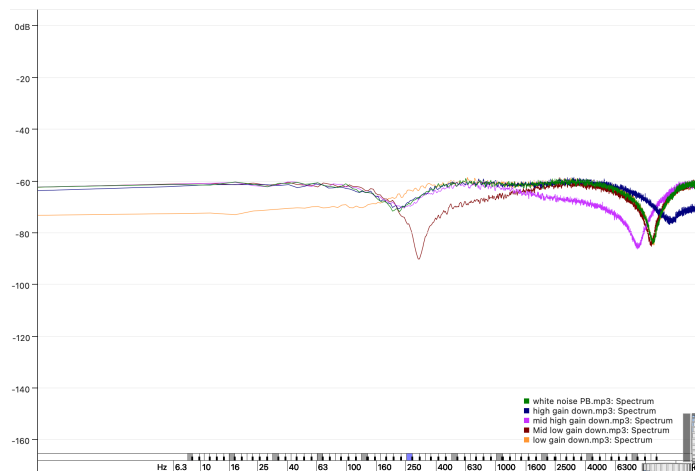


Figure 6: Layered spectra of different filter parameters, using the parallel band-splitting method

Conclusion: In the above report, I describe the process of creating 3 different plugins and discuss their respective qualities. In the first two implementations, the equaliser was constructed as a series of IIR filters, meaning the signal passed through each filter sequentially. By contrast, the third implementation constructed a parallel filter bank, whereby the input signal was split into multiple frequency bands using lowpass, bandpass, and highpass filters. Visual inspection of these spectra in Sonic Visualiser provided a clear demonstration of how different filter structures alter the frequency response. If I were to run this report again, I would experiment with making more bands (e.g up to 10) with different bandwidth ranges. I would also be interested in integrating these EQs into more complex plugins, such as within a delay or reverb unit.

References:

Reiss, J.D. and McPherson, A.P. (2015). *Audio effects theory, implementation and application*. Boca Raton London New York Crc Press, Taylor & Francis Group.